A Critical Code Review of the Web Witchcraft and Wizardry Project

Edmund Mulligan

April 22, 2026

Abstract

This essay presents a critical code review of the Web Witchcraft and Wizardry project, an interactive web-based learning platform. The review examines the project's modular architecture, focusing on JavaScript implementation including encrypted local storage, DOM manipulation, and defensive programming practices. The analysis demonstrates adherence to web development best practices while reflecting on the iterative development process and testing methodologies employed.

Contents

1 Introduction

This essay reviews the codebase of the Web Witchcraft and Wizardry project, a web-based interactive learning platform designed to teach users about web development concepts through a magical theme. The review covers the architecture and design of the project, as well as an analysis of the HTML, CSS, and JavaScript code, with an emphasis on the latter. Reflections on the development process and potential areas for improvement are also discussed. The code is available for review on GitHub at

`https://github.com/Birkbeck2/puffin-web-project-edmundmulligan`. Please review the code on the assessment-candidate branch. This will allow releases to the production web site to continue while the assessment is being conducted without changing the code being assessed. The deployed code can be viewed in GitHub Pages at

`https://birkbeck2.github.io/puffin-web-project-edmundmulligan`. Results of automated tests (along with design documentation) can be viewed at

`https://birkbeck2.github.io/puffin-web-project-edmundmulligan/diagnostics`.

2 Architecture and Design

The code has been designed so as to be maintainable and extensible, with a modular structure that separates concerns (Dijkstra, 1982). This is a front-end focused project, so the codebase consists of HTML, CSS, and JavaScript files, with a clear separation between structure, presentation, and behaviour (Keith and Sambells, 2010). Following DevOps principles (Allman, 2018), the project utilises a build pipeline to allow automated testing, as well as a version control system with multiple branches (development, staging, main and assessment-candidate) to manage changes.

Several constraints have been imposed on the project:

- Use only vanilla JavaScript, HTML, and CSS, without any external libraries or frameworks.
- No use of generative AI or Large Language Models (LLMs) in the development process.
- No server side code or databases.
- The website must be fully accessible and responsive, adhering to best practices in web development.
- Text intended for students must be below college-level reading ability, to ensure that the content is accessible to a wide audience.

3 HTML and CSS

The HTML code is well-structured and semantic, with appropriate use of elements to create a clear and logical document structure. The CSS code is organised into separate files for layout and components, with consistent naming conventions and a clear separation of concerns. The use of CSS variables allows for easy theming and customization of the website's appearance. The CSS has been completely refactored since the last assignment, to make it more maintainable and efficient. This includes keeping all global variables in definition files, using layers to control the cascade and specificity of styles, as well as removing redundant styles. Nested selectors have been used where possible to improve readability. At most three CSS files need to be loaded on any page (globals.css, main.css and a page specific CSS file if necessary), which makes it less likely a specific CSS file will be accidentally missed when developing new pages. The CSS code has been heavily influenced by tutorials from Kevin Powell (Powell, 2026b) and his new paid course (Powell, 2026a), which have provided valuable insights into modern CSS techniques and

best practices.

While most html files are reasonably well structured, students/lesson-00.html is an exception, as it is so large. As an HTML file it is unmaintainable and so has been generated from a mustache (Mustache, 2009) template and Javascript data file. There is no functionality in this web page that is not better illustrated in other more reasonably sized pages, so it is requested to be excluded from the assessment of this project.

4 Images

There is not much to say about the images used in the project, except that they are all optimised for web use and appropriately licensed. The gallery includes PNG, JPG and SVG images which allows the lesson on html images (lesson 4) to discuss the differences between these formats. The images are stored in separate folders which makes it easy to manage and reference them in the code (both on this website and in the code students will write using the images).

5 Javascript

The JavaScript code is organised into separate files for different components and features. The code follows best practices for readability and maintainability (McConnell, 2004), with consistent naming conventions, clear comments, and a focus on simplicity and self-documentation.

There is insufficient space here to discuss all the code in detail so this section highlights some of the more interesting and complex aspects. See Appendix A for a sample of examples of JavaScript code that demonstrate the use of various concepts and techniques required by this assessment.

5.1 localStorage.js

This is the most complex file in the codebase. It was written after a code review using CodeQL (GitHub, 2024) identified that using unencrypted local storage was a security risk. Arguably, no sensitive data is being stored so this is a false positive, but best practice is to eliminate security flaws before they become a problem (OWASP Foundation, 2010). It uses AES-GCM 256-bit encryption (National Institute of Standards and Technology, 2001) to encrypt data before storing it in localStorage, and decrypts it when retrieving. The encryption key is derived from a passphrase using PBKDF2 with a random salt and the encrypted data is stored as a JSON string containing the ciphertext and salt. This is not ideal as the passphrase is hard coded and in future could be improve by using a KeyVault (Microsoft Corporation, 2024) or similar service.

5.2 populateLineNumbers.js

This code is used to display code snippets on the website with line numbers. It works by selecting elements with the class 'code-snippet' or reading from a source file, splitting the text content into lines, and then creating a new HTML structure that includes both the line numbers and the code lines. The line numbers are generated dynamically based on the number of lines in the code snippet, and the original code is preserved in a separate element for styling purposes. This allows for separation of the structure of the code from its presentation, as well as making it easier to maintain and update the code snippets in the future. It also allows for the snippets to be easily copied by students without including the line numbers.

5.3 injectGlossaryPopover.js

This fetches glossary data from glossary.html and injects it into the page as popovers that appear when clicking on an icon next to glossary terms. It works by selecting all elements with the class 'glossary-term', retrieving the corresponding definition from the glossary data, and then creating a popover element that is displayed on hover. To prevent injection attacks, the code sanitises the glossary definitions by escaping any HTML tags before inserting them into the popover. This ensures that any potentially malicious code in the glossary definitions cannot be executed when the popover is displayed. Rendering of the popover is handled by CSS, which, as usual, allows for separation of structure and presentation.

5.4 utils.js, queryParameters.js and debug.js

These scripts are not directly related to any specific features of the website, but provide utility functions and debugging tools that are used throughout the codebase. The utils.js file contains a variety of helper functions for tasks such as event handling and data formatting. The queryParameters.js file provides a simple interface for working with URL query parameters, allowing for automated testing of different scenarios. The debug.js file contains functions for logging and displaying debug information, which was essential during development and testing.

6 Reflections

The development process was highly iterative, with continuous refinement driven by automated testing and manual code review. Given the scope of the project, and the fact that it was developed by a single person, this was the only feasible approach and drew on my considerable experience of developing in a DevOps environment. The website is at the limit of what can be achieved without server side processing. In particular, using local storage to persist data is inferior to using a database - if users switch computers their saved data will be lost and gamification (Dicheva et al., 2015), such as progress tracking and leader boards, is not possible.

Although not the focus of this assignment, I am particularly pleased with how the CSS refactoring turned out. The CSS submitted last term was functional but messy, reflecting the fact that I was learning it while developing the project. The refactored version is much cleaner, particularly with the use of CSS variables in the definitions folder and layers. The Javascript code is reasonably well organised but could be improved. For example localStorage.js could and should, with more time, be refactored into classes to handle encryption and storage separately.

7 Conclusion

This code review demonstrates that the Web Witchcraft and Wizardry project achieves its academic objectives through well-structured, maintainable code adhering to web development best practices. It is at a level of maturity where it can be given to students, mentors and teachers as a working prototype to solicit feedback before being released as a final product. The deliberate architectural choices (modular JavaScript organization, semantic HTML, modern CSS and separation of concerns) create a robust foundation for future enhancement. The comprehensive testing regime validates accessibility and standards compliance. The codebase exhibits professional-grade practices including encryption for data storage and defensive programming against injection attacks. Further work is required to write the lessons and this will be the focus of the next phase of development, along with user and expert testing.

Word count

Word count: 1525 words (excluding references)

References

Eric Allman. Devops: Software architects' perspectives. *Communications of the ACM*, 61(4): 40–44, 2018. doi: 10.1145/3186331.

Darina Dicheva, Christo Dichev, Gennady Agre, and Galia Angelova. Gamification in education: A systematic mapping study. *Educational Technology & Society*, 18(3):75–88, 2015.

Edsger W. Dijkstra. On the role of scientific thought. In *Selected Writings on Computing: A Personal Perspective*, pages 60–66. Springer, New York, 1982. Originally published in 1974. Introduces the principle of separation of concerns.

GitHub. CodeQL: The code analysis engine for finding security vulnerabilities, 2024. URL `https://codeql.github.com/`. Accessed: 2026-04-12.

Jeremy Keith and Jeffrey Sambells. *DOM Scripting: Web Design with JavaScript and the Document Object Model*. Apress, Berkeley, CA, 2nd edition, 2010. Advocates for unobtrusive JavaScript and separation of structure (HTML), presentation (CSS), and behavior (JavaScript).

Steve McConnell. *Code Complete: A Practical Handbook of Software Construction*. Microsoft Press, 2nd edition, 2004. ISBN 978-0735619678.

Microsoft Corporation. Azure key vault: Safeguard cryptographic keys and other secrets. Technical documentation, Microsoft Corporation, 2024. URL `https://learn.microsoft.com/en-us/azure/key-vault/`.

Mustache. Mustache: Logic-less templates. `https://mustache.github.io/`, 2009. Accessed: April 2026.

National Institute of Standards and Technology. Advanced encryption standard (aes). Technical Report FIPS PUB 197, National Institute of Standards and Technology, Gaithersburg, MD, 2001. URL `https://doi.org/10.6028/NIST.FIPS.197`.

OWASP Foundation. Secure coding practices - quick reference guide, 2010. URL `https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/`. Accessed: 2026-04-12.

Kevin Powell. Css demystified. Online course, 2026a. URL `https://courses.thecascade.dev/course/css-demystified/`. Paid course. Accessed: April 2026.

Kevin Powell. Kevin powell youtube channel, 2026b. URL `https://www.youtube.com/@KevinPowell`. Accessed April 2026.

Appendices

A Javascript Assessment Criteria

Here are examples of Javascript concepts used in the codebase to allow assessors to easily identify where key features are implemented. This is not an exhaustive list of all Javascript concepts used in the codebase, but rather a quick guide to where an instance of each key feature can be found. Assessors are, of course, free to review the codebase in its entirety to gain a deeper understanding of the implementation and design choices made throughout the project.

Variables and Functions — animatePortrait.js

setRandomPortraitPosition (container) is a method (function) that uses variables to store the position of the portrait and the dimensions of the container. It also calls JavaScript's built-in Math.random () function to generate random positions for the portrait.

Objects and Arrays — animatePortrait.js

PortraitAnimator class is instantiated as an object and uses an array to store the portrait images that are animated.

Conditional Logic and Loops — animateWand.js

Uses a foreach loop to iterate over path elements in svg base element and a conditional to deal with the possibility that the svg file does not contain a base svg element and to determine if a path element should be animated with a sparkle.

DOM Manipulation — animatePortrait.js

The DOM is manipulated in method setPortraitImage (container, image) to change the source of the portrait image and update the alt text.

Interactivity — toggleSection.js

toggleSectionCollapse (sectionId, event) is a method that toggles the collapse state of a section in response to events (e.g. clicking a button). It uses event listeners to detect user interactions and updates the DOM to show or hide the content of the section.

Data Storage — localStorage.js

The LocalStorage class provides methods for storing encrypted data in the browser's local storage. This allows the application to persist user preferences across pages and sessions.

Data Retrieval — injectGlossaryPopovers.js

The injectGlossaryPopovers () method retrieves glossary terms and their definitions using fetch () to make an HTTP request to the glossary page of the website. It then processes the response to extract the relevant data and injects it into the DOM as popovers. This technique decouples the glossary data from the lessons which render the popovers, allowing for easier maintenance and updates to the glossary without needing to modify the lesson code, while ensuring the lessons always display the most up-to-date glossary definitions.